

# SLAM performance prediction

Mauro Tellaroli

Statistical Methods for Machine Learning

Università degli Studi di Milano

*I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.*

## Contents

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                            | <b>2</b> |
| 1.1      | SLAM methods . . . . .                         | 2        |
| 1.2      | Accuracy of SLAM methods . . . . .             | 2        |
| 1.3      | APE calculation . . . . .                      | 2        |
| 1.4      | Project goal . . . . .                         | 3        |
| <b>2</b> | <b>Data</b>                                    | <b>4</b> |
| 2.1      | Dataset composition . . . . .                  | 4        |
| 2.2      | Data preprocessing . . . . .                   | 4        |
| 2.3      | Data augmentation . . . . .                    | 6        |
| <b>3</b> | <b>Models</b>                                  | <b>6</b> |
| 3.1      | AlexNet . . . . .                              | 6        |
| 3.1.1    | Dropout . . . . .                              | 7        |
| 3.2      | MobileNetV3 . . . . .                          | 7        |
| 3.2.1    | Depthwise and Pointwise Convolutions . . . . . | 7        |
| 3.2.2    | Squeeze and Excitation . . . . .               | 8        |
| 3.2.3    | HardSwish Activation function . . . . .        | 9        |
| 3.3      | SqueezeNet . . . . .                           | 10       |
| 3.3.1    | Fire modules . . . . .                         | 10       |

# 1 Introduction

In the field of autonomous mobile robot, the Simultaneous Localization And Mapping (SLAM) represents one of the most significant problems. As the name suggests, the goal of the SLAM is incrementally obtain from a mobile robot a map of the environment and simultaneously locate the robot's position inside it. The difficulty of the SLAM problem is that to precisely localize itself, the robot needs an accurate map, while to obtain an accurate map, it needs an accurate localization inside the map. This type of problems is called chicken-and-egg problem.

## 1.1 SLAM methods

In the last 30 years several approaches have been developed to solve the SLAM problem leading to methods extensively used in both common and industrial scenarios. The main paradigms [1] used are:

- Extended Kalman Filter: a vector containing the estimates of the robot position and the environment landmark is used. In addition, a matrix composed of the correlations between the positions and landmark estimates is also used. Both the vector and the matrix are updated using the extended Kalman Filter.
- Particle Filter: the Monte Carlo method is used to sample the possible states of the robot. A random sample is called a particle and it's composed of the robot position and a map estimate. The main steps are: predict the robot's pose for each particle, update particle weights based on sensor data, resample particles according to their weights, and update each particle's map.
- Graph-based: a graph is used where nodes represent robot poses or landmarks, and edges represent spatial constraints derived from sensor observations or odometry. The SLAM problem is formulated as a graph optimization task, where the goal is to find the configuration of nodes that best satisfies all constraints, typically using nonlinear optimization techniques.

## 1.2 Accuracy of SLAM methods

The heterogeneity of SLAM methods makes it difficult to find a measure that can evaluate their godness. Many approaches rely on manual evaluation or utilize information derived from the specific algorithm. Nowadays the most used metric is the Absolute Pose Error (APE)[2]. The APE measures the difference between the estimated trajectory produced by the SLAM algorithm and the ground truth trajectory, focusing on the global consistency of the estimated poses. The big advantage of the APE is that it allows for a quantitative assessment of how well the SLAM algorithm reconstructs the robot's trajectory over time, independent of the internal workings of the specific method used.

The APE is composed of two sub-metrics:

1. The Absolute Translational Error (ATE): it measures the difference in position, typically using the Euclidean distance between corresponding poses.
2. The Absolute Rotational Error (ARE): it measures the difference in orientation, often computed as the angular distance between corresponding rotations.

Figure 1 and 2 show respectively the ATE variation over time and the difference between the ground truth and the robot estimated trajectory during an environment exploration.

## 1.3 APE calculation

As anticipated in the previous section, the APE metric is based on the difference between the estimated robot poses and the corresponding ground truth poses.

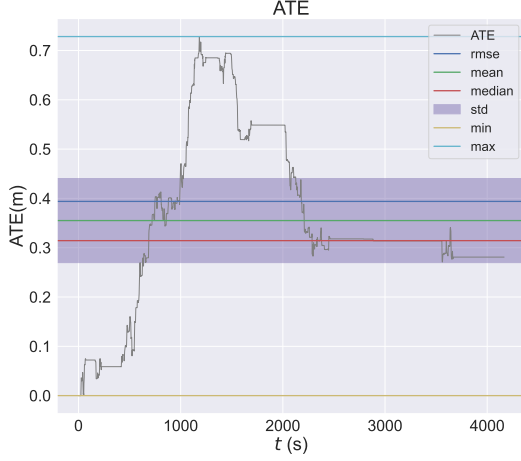


Figure 1: ATE variation over time

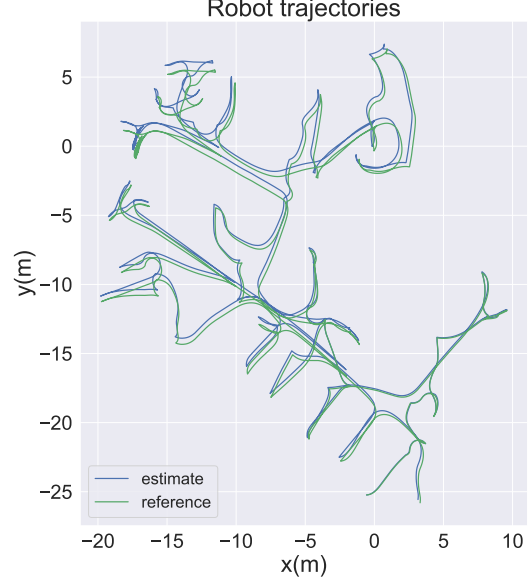


Figure 2: Difference between real (in green) and estimated (in blue) trajectory

Formally, let  $\mathbf{x}_{1:T}$  be the poses of the robot estimated by a SLAM algorithm from time step 1 to  $T$ , and let  $\mathbf{x}_{1:T}^*$  be the corresponding ground truth poses.

Let  $\delta_{i,j}$  be the relative transformation that moves the robot from pose  $x_i$  to  $x_j$ :

$$\delta_{i,j} = x_j \ominus x_i$$

$$\delta_{i,j}^* = x_j^* \ominus x_i^*$$

Let  $trans(\cdot) \in \mathbb{R}^2$  be the function that extracts the translational (position) component from a transformation, and let  $rot(\cdot) \in \mathbb{R}^2$  extract the rotational (orientation) component as Euler angle.

Finally, to compute the APE:

$$APE = \underbrace{\frac{1}{N} \sum_{i,j} trans(\delta_{i,j} \ominus \delta_{i,j}^*)^2}_{ATE} + \underbrace{\frac{1}{N} \sum_{i,j} rot(\delta_{i,j} \ominus \delta_{i,j}^*)^2}_{ARE}$$

## 1.4 Project goal

Typically, the APE is calculated a posteriori, after the robot has explored the entire environment. The main goal of this project is to propose a predictor for the SLAM performance, specifically the APE, before running the algorithm or during its execution, using features derived from the input data or intermediate results. This type of prediction can improve the robustness and efficiency of autonomous mobile systems since the position error (APE) can be used to correct the robot trajectory.

The work in [3] proposes two main APE predictors, one based on linear regression, and another one based on a Gaussian Process via an RBF Kernel. Although Gaussian Process regression showed slightly better performance compared to linear regression model, the authors preferred linear regression because they considered it a more interpretable model. In both cases, a feature extractor is used [4] obtaining a set of structural features to represent the environment as a vector, which is then passed to the model.

## 2 Data

### 2.1 Dataset composition

The dataset is composed of 4613 datapoints. Each datapoint is represented as a tuple

$$(M, a, ate, are)$$

where:

1.  $M \in [0, 255]^{n \times n}$ : floorplan images representend as grayscale maps that a robot has obtained during its navigation. Some examples are reported in Figure 3;
2.  $a \in \mathbb{R}^+$ : area measurements of the floorplan images, expressed in  $m^2$ ;
3.  $ate \in \mathbb{R}^+$ : ATE values, expressed in  $m$ ;
4.  $are \in \mathbb{R}^+$ : ARE values, expressed in  $rad$ ;

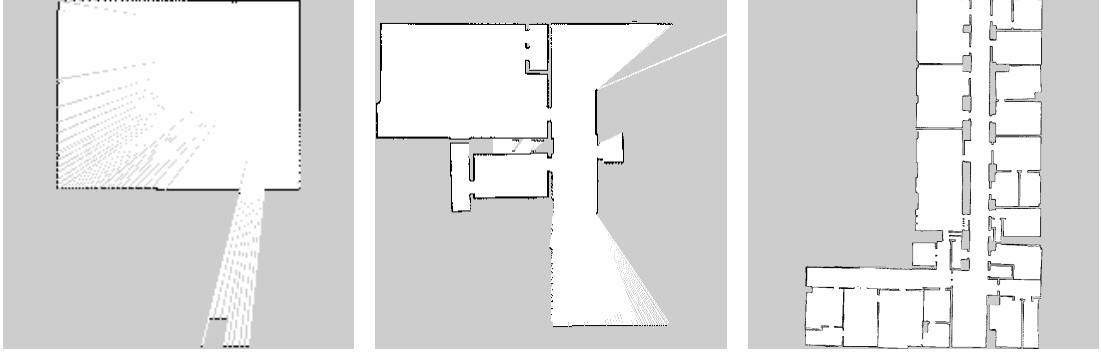


Figure 3: Examples of grayscale floorplan images used as input; the white pixels represent the free area, the black pixels the obstacles and the gray pixels the unknown area.

### 2.2 Data preprocessing

Evaluating the APE values shows that they are mainly distributed towards zero, with a significant presence of outliers, especially for ATE values. Figure 4 shows the box plot with highlighted the 0.99-quantile and the first value such that its Z-score is greater or equal to 3.5; the Z-score of a number  $x$  is  $Z = \frac{x - \bar{x}}{\sigma}$  where  $\bar{x}$  is the mean and  $\sigma$  the standard deviation.

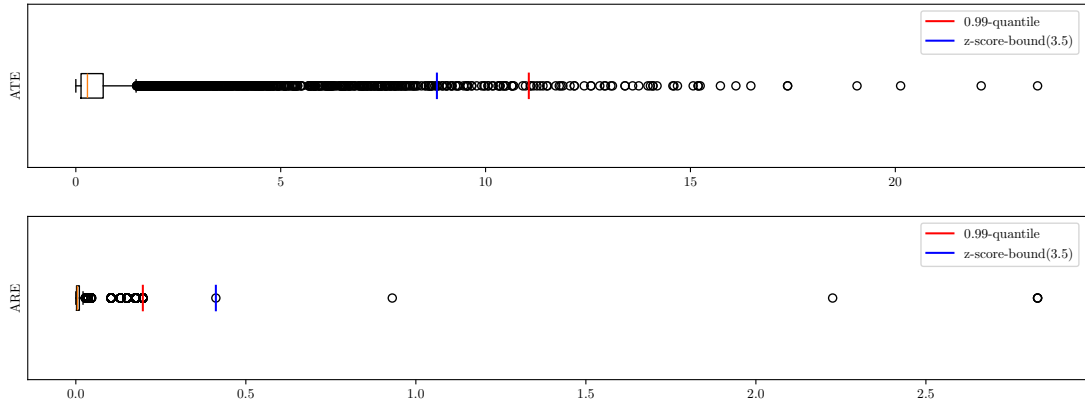


Figure 4: Box plot of the ATE (top) and ARE (bottom) values; is also shown the 0.99-quantile (red line) and the first value with a z-score equal to 3.5 (blue line).

To facilitate our models to learn each quantity, the following strategies have been applied:

- Outliers cleaning: data points with ATE or ARE values above the 0.99-quantile or with a Z-score greater than or equal to 3.5 have been removed to reduce the impact of extreme outliers. 113 datapoints were removed;
- Logarithmic rescaling: due to the skewed distribution of ATE and ARE, a logarithmic transformation has been applied to these values to reduce the effect of large values and improve model learning; let  $x$  be the interested value to be rescaled, the applied operation is:

$$\log(1 + x)$$

where log is the natural logarithm;

- Rescaling: after the logarithmic rescaling, a linear rescaling to  $[0, 1]$  has been applied; let  $\{x_1, \dots, x_n\}$  be interested values to be rescaled, the applied operation is

$$\frac{x_i}{\max_{1 \leq k \leq n} x_k} \quad i=1, \dots, n$$

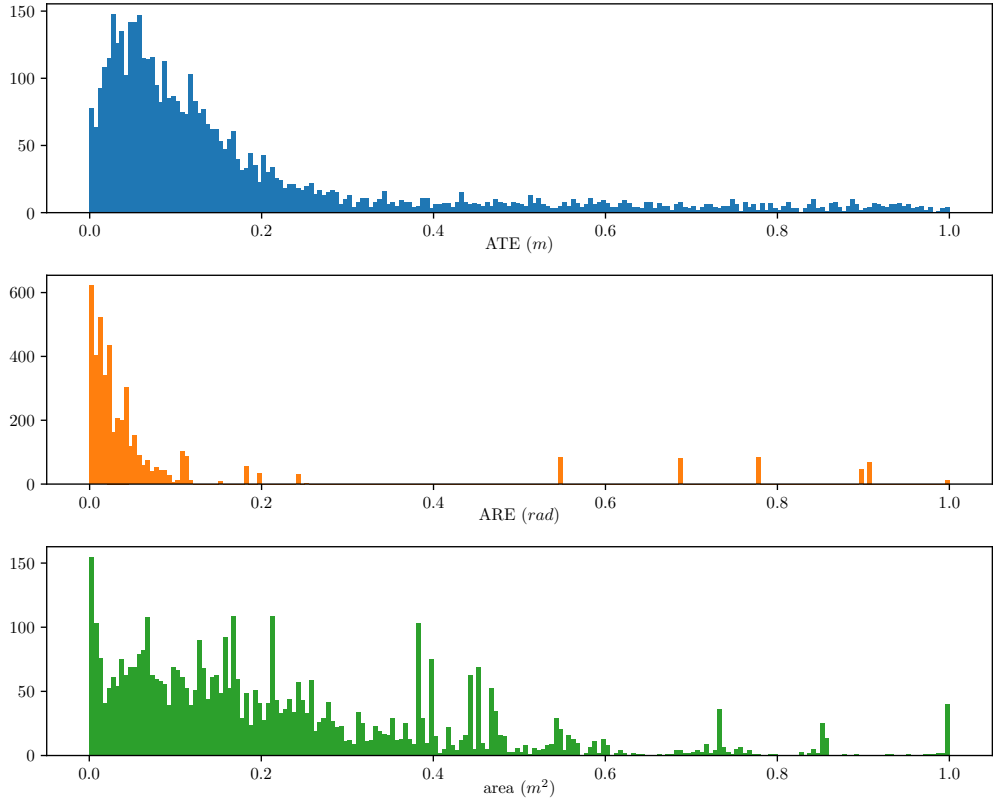


Figure 5: Dataset values after the log and linear rescaling.

- Image cropping: each map has been cropped to remove unnecessary borders, focusing only on the relevant area to reduce input size and eliminate irrelevant background; the crop is always squared in order to maintain this property to all floorplan images. Figure 6 show an example of this operation;



Figure 6: A floorplan (left) and its bounding square crop (right).

- Image resizing: each floorplan image is resized to  $400 \times 400$  pixels to ensure a uniform input size for the neural network models;

### 2.3 Data augmentation

Given the relatively limited size of the dataset, the following data augmentation techniques have been applied to training set:

- Random flip: both horizontal and vertical flips are applied;
- Random rotation: random rotation between  $0^\circ, 90^\circ, 180^\circ, 270^\circ$  is applied;

## 3 Models

In this project, different architectures are used, all based on Convolutional Neural Networks (CNNs) to delegate feature extraction to the model, directly using the environment floorplan image as input. Three distinct models were used:

1. AlexNet [5], one of the first successful architectures in the field of CNNs;
2. MobileNetV3 [6], the state of the art of lightweight CNN architectures designed for mobile and embedded vision applications;
3. SqueezeNet [7], an ultra-compact CNN with few parameters, offering decent accuracy for low-memory device;

Formally, the model  $h$  we aim to learn can be expressed as follows:

$$h : [0, 255]^{n \times n} \times \mathbb{R}^+ \rightarrow \mathbb{R}^2$$

where  $[0, 255]^{n \times n}$  is a floorplan image encoded as an  $n \times n$  grayscale image,  $\mathbb{R}^+$  is the area of the floorplan expressed in  $m^2$ , and  $\mathbb{R}^2$  contains ATE ( $m$ ) and ARE ( $rad$ ).

### 3.1 AlexNet

To begin, a model based on AlexNet was chosen, one of the first successful architectures in the field of convolutional neural networks. AlexNet consists of several convolutional layers followed by fully connected layers, using ReLU activations and max-pooling. The relatively large number of layers without the use of modern techniques to lighten its computational load, makes this model a deep CNN resulting in a high number of parameters (37 298 882). The abstract structure is shown on Figure 7.

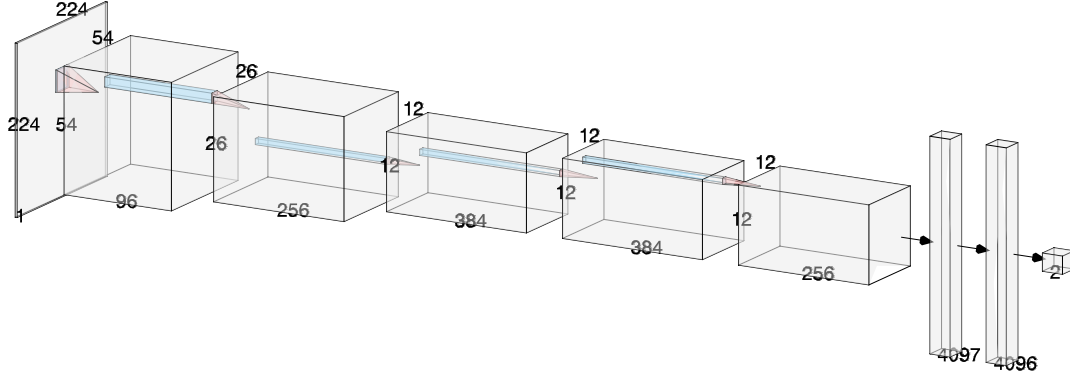


Figure 7: Model structure based on AlexNet. Realized with [8].

Two main sections can be outlined:

1. A CNN feature extractor: it processes the input image through a series of convolutional and pooling layers, resulting in a condensed set of features;
2. A Regression head: it predicts ATE and ARE values through some fully connected layers whose input is the set of features extracted from the CNN;

Note that the first layer of the regression head has size 4097 instead of 4096, since the area of the floorplan is added as an input concatenated to the feature vector extracted by the CNN. This is because otherwise the scaling done by the model in the initial stage would cause the true value of the area to be lost. This procedure was done on all three proposed models.

### 3.1.1 Dropout

To reduce overfitting, the dropout technique [9] is applied within the fully connected layers of the regression head. Dropout randomly sets a fraction of the input units to zero during training, which prevents the network from relying too heavily on specific features and encourages the learning of more robust representations. In our implementation, a dropout rate of 0.5 is used after each fully connected layer.

## 3.2 MobileNetV3

Despite AlexNet’s good accuracy, the computational requirements it needs make it unsuitable for hardware used in mobile robotics. For this reason, MobileNetV3 was selected as a more efficient alternative, offering a state-of-the-art trade-off between accuracy and computational efficiency. In particular, the MobileNetV3 small version was used, which has 1 520 642 parameters, 24 times less than AlexNet.

To achieve this result, different optimization techniques are used, but we will focus only on the main ones.

### 3.2.1 Depthwise and Pointwise Convolutions

Convolutional layers on which CNNs are based have led to massive achievements in machine learning projects, especially in computer vision. However, standard convolutions are computationally expensive, particularly for deep networks. MobileNetV1 addresses this by employing depthwise separable convolutions, which split the convolution operation into two parts:

1. Depthwise convolution: processes each input channel separately with its own filter, extracting spatial features independently for each channel;
2. Pointwise convolution: applies  $1 \times 1$  convolutions across all channels, combining the outputs from the depthwise step to integrate information across channels;

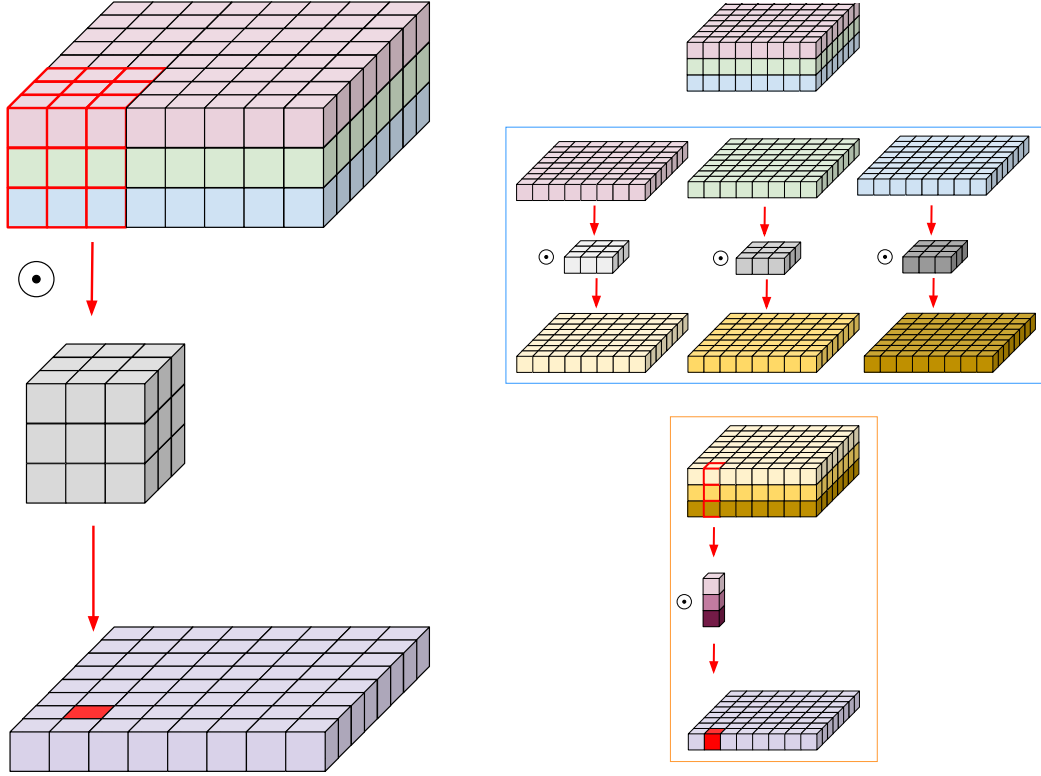


Figure 8: Standard Convolution (left) and Depthwise+Pointwise Convolution (right). The Depthwise conv. is highlighted in light blue while Pointwise conv. in orange [10].

To understand computational savings, take the following example: let  $S$  be the dimensions of the squared input,  $K$  the dimensions of the squared kernel filter,  $C_{in}$  the number of input channels and  $C_{out}$  the numbers of output channels. We'll assume:

$$S = 128 \quad K = 3 \quad C_{in} = 3 \quad C_{out} = 16$$

|                   | Regular convolution             | Depthwise and Pointwise convolution                              |
|-------------------|---------------------------------|------------------------------------------------------------------|
| Parameters        | $3^2 \cdot 3 \cdot 16 = 432$    | $3^2 \cdot 3 \cdot 128^2 \cdot 16 = 7\,077\,888$                 |
| # multiplications | $3^2 \cdot 3 + 3 \cdot 16 = 75$ | $3^2 \cdot 3 \cdot 128^2 + 128^2 \cdot 3 \cdot 16 = 1\,228\,800$ |

### 3.2.2 Squeeze and Excitation

Introduced in [11], Squeeze and Excitation is a technique that focuses on the interdependencies between channels. As the name suggests, there are two phases:

1. Squeeze phase: each channel is squeezed into a single value through the use of average pooling. The output of this phase is then a vector  $\mathbf{z}$  of  $C$  elements, where  $C$  is the number of channels:

$$\mathbf{z} = \{z_1, \dots, z_C\}$$

$$z_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W u_c(i, j) \quad c=1, \dots, C$$

where  $\mathbf{U} = \{\mathbf{u}_1, \dots, \mathbf{u}_C\}$  is the set of feature maps of size  $H \times W$  and  $u_c(i, j)$  is the element in position  $(i, j)$  on the  $c$ -th feature map.

2. Excitation phase: the squeezed vector  $\mathbf{z}$  is fed into a two-layer feed-forward neural network composed of a first layer of size  $C/r$  with ReLU and a second layer of size  $C$  with sigmoid,



where  $\text{sigmoid}(x) = 1/(1 + e^{-x})$  and  $r$  is a reduction ratio. The purpose of this phase is to capture the intricate dependencies among channels. The output of this network is another vector  $\mathbf{s}$  of the same dimensions of  $\mathbf{z}$ , containing the importance weights for each channel.

3. Scale and combine phase: the importance weights vector  $\mathbf{s}$  is used to rescale each channel of the original input by channel-wise multiplication (\*):

$$\tilde{\mathbf{x}}_c = s_c * \mathbf{u}_c \quad c=1, \dots, C$$

where  $\tilde{\mathbf{X}} = \{\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_C\}$  is the final output. In this way most important feature are highlighted and the less important ones are downplayed.

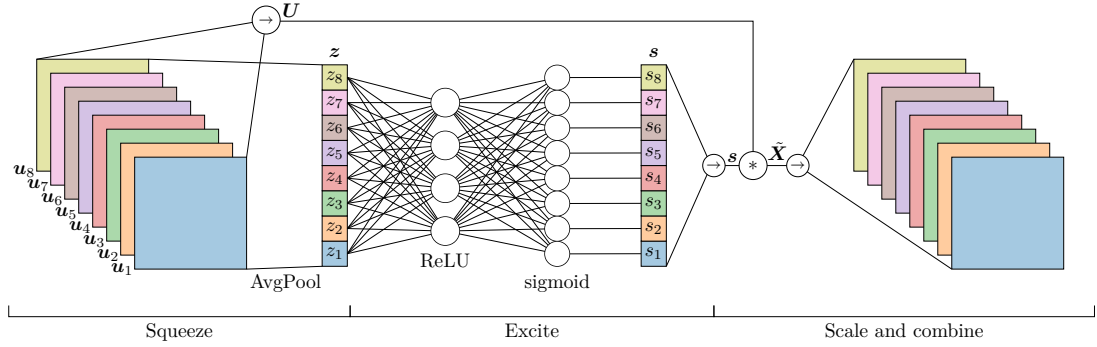


Figure 9: A graphical representation of Squeeze and excitation; in this case a reduction ratio of  $r = 2$  is used with 8-channel feature maps.

### 3.2.3 HardSwish Activation function

To improve the accuracy of the neural networks ReLU was replaced by a new activation function called *swish*, defined as:

$$\text{swish } x = x \cdot \sigma(x)$$

where  $\sigma$  is the sigmoid function  $\sigma(x) = 1/(1 + e^{-x})$ . However, despite swish improves accuracy, it is computationally expensive for mobile devices. To address this, MobileNetV3 introduces the *HardSwish* activation function, which approximates swish while being more efficient to compute. HardSwish is defined as:

$$\text{h-swish}(x) = x \cdot \frac{\text{ReLU6}(x + 3)}{6}$$

where  $\text{ReLU6}(x) = \min(\max(0, x), 6)$ . This function retains the benefits of swish but is faster and more suitable for deployment on resource-constrained hardware.

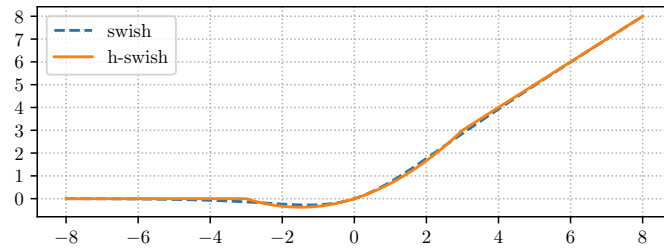


Figure 10: Swish and h-swish

### 3.3 SqueezeNet

As final model SqueezeNet was chosen. SqueezeNet is an ultra-compact convolutional neural network architecture designed to achieve AlexNet-level accuracy with significantly fewer parameters, as the original paper title “AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size” [7] suggests; as proof of this, the model that was used, derived from a minor modification of SqueezeNet v1.1, has 722 740 params,  $\sim 51.6$  times less than AlexNet models described in section 3.1. The few parameters of SqueezeNet also led to a large reduction in power consumption and inference time, making it one of the most widely used architectures in various fields such as autonomous driving, IoT, and mobile AI.

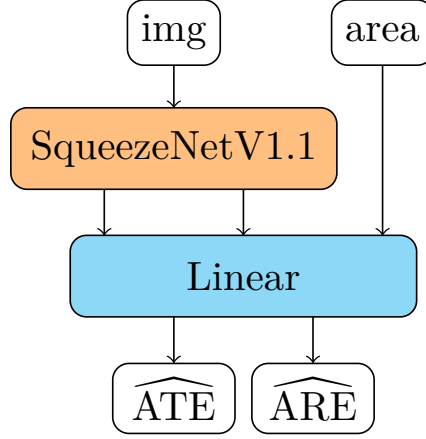


Figure 11: .

Consider a convolutional layer formed only with  $3 \times 3$  filters. Its numbers of parameters is:

$$(\# \text{ filters}) \cdot (\# \text{ input channels}) \cdot (3 \cdot 3)$$

So, to maintain a small number of parameters in a CNN it is important to reduce both the number of filters and the number of input channels. The authors of [7], following this reasoning, have employed three strategies:

1. Replace most of  $3 \times 3$  filters with  $1 \times 1$  filter: in this way they reduce the numbers of  $3 \times 3$  filters by replacing them with  $1 \times 1$  filters that have 9 times fewer parameters;
2. Decrease the number of input channels of  $3 \times 3$  filters: to do so squeeze layers was used (similar but different from the squeeze phase described in section 3.2.2);
3. Downsample late in the network: in this way convolutional layers have large activation map leading to higher accuracy [12];

To employ this three strategies Fire modules are introduced.

#### 3.3.1 Fire modules

A Fire module consists of:

- A squeeze convolution layer: it uses  $s_{1 \times 1}$  filters  $1 \times 1$  to squeeze all the input channels into  $s_{1 \times 1}$  single channel feature maps (Figure 12);
- An expand layer: formed by  $e_{1 \times 1}$  filters  $1 \times 1$  and  $e_{3 \times 3}$  filters  $3 \times 3$ ; it takes as input the  $s_{1 \times 1}$  feature maps and returns the  $e_{1 \times 1} + e_{3 \times 3}$  feature maps obtained from the convolutional filters of this layer. To limit the number of input channels of the  $3 \times 3$  filters, Fire modules set  $s_{1 \times 1}$  to be less than  $e_{1 \times 1} + e_{3 \times 3}$ .

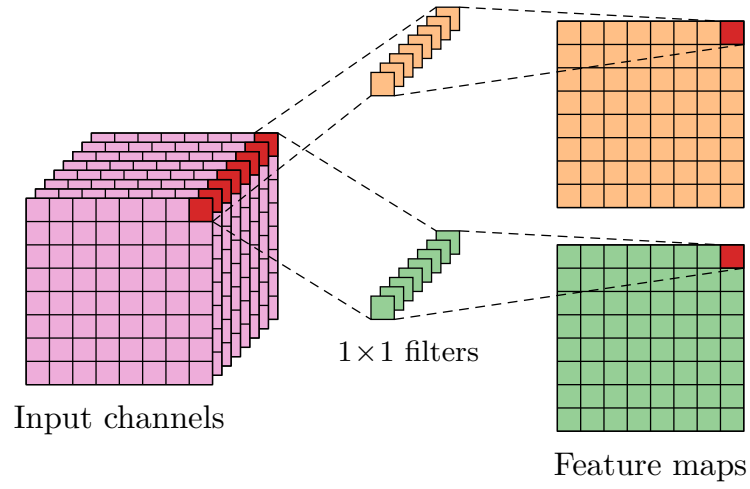


Figure 12: Squeeze convolutional layer with  $s_{1 \times 1} = 2$  and 8 input channels.

By stacking multiple fire modules, SqueezeNet achieves a significant reduction in the number of parameters while maintaining competitive accuracy.

## References

- [1] Cyrill Stachniss, John J. Leonard, and Sebastian Thrun. “Simultaneous Localization and Mapping”. In: *Springer Handbook of Robotics*. Ed. by Bruno Siciliano and Oussama Khatib. Cham: Springer International Publishing, 2016, pp. 1153–1176. ISBN: 978-3-319-32552-1. DOI: 10.1007/978-3-319-32552-1\_46. URL: [https://doi.org/10.1007/978-3-319-32552-1\\_46](https://doi.org/10.1007/978-3-319-32552-1_46).
- [2] Rainer Kümmerle et al. “On Measuring the Accuracy of SLAM Algorithms”. In: *Autonomous Robots* 27 (Nov. 2009), pp. 387–407. DOI: 10.1007/s10514-009-9155-6.
- [3] Matteo Luperto, Valerio Castelli, and Francesco Amigoni. “Predicting Performance of SLAM Algorithms”. In: *CoRR* abs/2109.02329 (2021). arXiv: 2109.02329. URL: <https://arxiv.org/abs/2109.02329>.
- [4] Matteo Luperto and Francesco Amigoni. “Extracting Structure of Buildings Using Layout Reconstruction”. In: *Intelligent Autonomous Systems 15*. Ed. by Marcus Strand et al. Cham: Springer International Publishing, 2019, pp. 652–667. ISBN: 978-3-030-01370-7.
- [5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012.
- [6] Andrew Howard et al. *Searching for MobileNetV3*. 2019. arXiv: 1905.02244 [cs.CV]. URL: <https://arxiv.org/abs/1905.02244>.
- [7] Forrest N. Iandola et al. *SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size*. 2016. arXiv: 1602.07360 [cs.CV]. URL: <https://arxiv.org/abs/1602.07360>.
- [8] URL: <https://alexlenail.me/NN-SVG/AlexNet.html>.
- [9] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.
- [10] URL: <https://eli.thegreenplace.net/2018/depthwise-separable-convolutions-for-machine-learning/>.
- [11] Jie Hu et al. *Squeeze-and-Excitation Networks*. 2019. arXiv: 1709.01507 [cs.CV]. URL: <https://arxiv.org/abs/1709.01507>.
- [12] Kaiming He and Jian Sun. *Convolutional Neural Networks at Constrained Time Cost*. 2014. arXiv: 1412.1710 [cs.CV]. URL: <https://arxiv.org/abs/1412.1710>.